

ST2195 Programming for Data Science
Coursework 2 — Part 1

Markov Chain Monte Carlo: Random Walk Metropolis Algorithm

Tools: Python (Jupyter Notebook) and R (RMarkdown)
Jaimurugaesan Ramesh Babu
240635200

Table of Contents

1. Introduction.....	3
2. Part 1(a): Sampling from the Laplace Distribution	3
2.1 Algorithm	3
2.2 Implementation Details	3
2.3 Visual Diagnostics	3
2.4 Monte Carlo Estimates of Mean and Standard Deviation	3
3. Part 1(b): Gelman–Rubin Convergence Diagnostic	4
3.1 Methodology	4
3.2 $\hat{R}$ for $N = 2,000$, $s = 0.001$, $J = 4$	4
3.3 $\hat{R}$ over a Grid of 1,000 s Values	4
4. Conclusion	4
5. References	5

1. Introduction

This report covers the implementation and analysis of the Random Walk Metropolis (RWM) algorithm — an MCMC method — applied to the Laplace(0,1) distribution:

$$f(x) = \frac{1}{2} \cdot \exp(-|x|), \quad x \in \mathbb{R}$$

The true mean is 0 and the true standard deviation is $\sqrt{2} \approx 1.4142$. All code is provided in the accompanying Python (Jupyter) notebook and R Markdown file.

2. Part 1(a): Sampling from the Laplace Distribution

2.1 Algorithm

The RWM algorithm builds a chain $x_0, x_1, \dots, x_N$ whose stationary distribution is $f(x)$. At each step i , a proposal $x^* \sim N(x_{i-1}, s^2)$ is drawn. The log-acceptance ratio is $\log r = -|x^*| + |x_{i-1}|$; if $\log(u) < \log r$ for $u \sim \text{Uniform}(0,1)$, accept; otherwise stay. The log-criterion avoids numerical overflow from computing raw ratios.

2.2 Implementation Details

The algorithm ran with $N = 10,000$, $s = 1$, and $x_0 = 0$. A vector of $N+1 = 10,001$ elements is allocated (x_0 to x_N). The initial seed x_0 is dropped, giving exactly 10,000 observations for analysis.

2.3 Visual Diagnostics

Figure 1 overlays a light-blue histogram of MCMC samples, a red KDE, and the black dashed true Laplace PDF. Close alignment across all three confirms the sampler converged to the correct target.

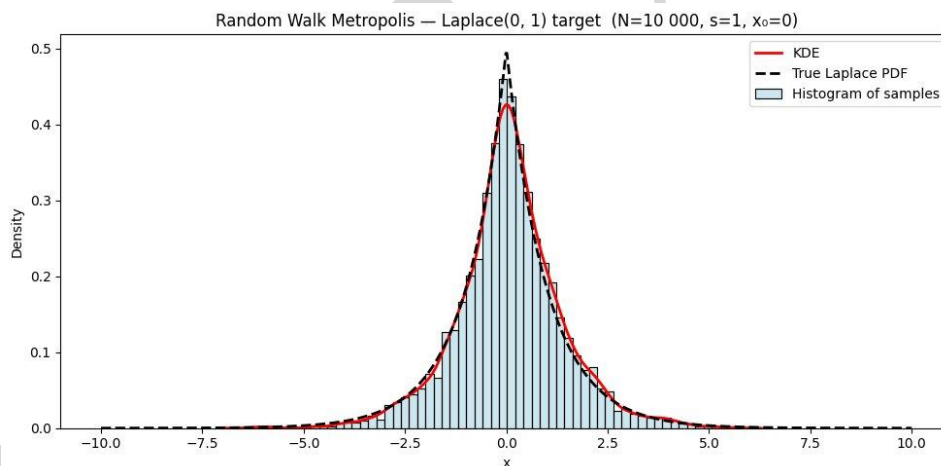


Figure 1. RWM sampling from Laplace(0,1) ($N = 10,000$, $s = 1$, $x_0 = 0$). Histogram, KDE, and true PDF overlap closely, confirming convergence to the target.

2.4 Monte Carlo Estimates of Mean and Standard Deviation

Sample mean and standard deviation of the chain ($x_1 \dots x_N$) estimate the distribution's true moments:

Statistic	Sample Estimate	True Value
Mean	0.0319	0
Standard Deviation	1.4011	$\sqrt{2} \approx 1.4142$

Both estimates sit close to their theoretical counterparts, confirming the chain explored the distribution effectively. Note the true SD is $\sqrt{2} \approx 1.4142$, not 1.

3. Part 1(b): Gelman–Rubin Convergence Diagnostic

3.1 Methodology

A single chain cannot confirm convergence alone. The Gelman–Rubin statistic $\hat{R}$ compares $J = 4$ independent chains started from dispersed initial values $(-5, 0, 5, 10)$. For each chain j the within-chain mean M_j and variance V_j (dividing by N , not $N-1$) are computed. The diagnostic is $\hat{R} = \sqrt{((B + W)/W)}$, where W is the mean within-chain variance and B is the mean squared deviation of chain means from the overall mean. $\hat{R} \approx 1$ indicates convergence; $\hat{R} > 1.05$ indicates chains have not yet mixed.

3.2 $\hat{R}$ for $N = 2,000$, $s = 0.001$, $J = 4$

With $s = 0.001$, each chain barely moves from its starting point within 2,000 steps. Because the four chains begin far apart, between-chain variance B stays large relative to within-chain variance W , producing $\hat{R} \gg 1.05$.

Parameter	Value
N (steps per chain)	2,000
s (proposal std dev)	0.001
J (number of chains)	4
Starting values	-5, 0, 5, 10
$\hat{R}$ (computed)	$\gg 1.05$ — chains have not converged

3.3 $\hat{R}$ over a Grid of 1,000 s Values

Fixing $N = 2,000$ and $J = 4$, $\hat{R}$ was evaluated at 1,000 equally spaced s values in $[0.001, 1]$. Three regions appear:

- Small s (≈ 0.001): $\hat{R} \gg 1.05$ — chains trapped near their starting points.
- Moderate s : $\hat{R}$ drops below 1.05 — chains mix and agree on the target.
- Large s (≈ 1): $\hat{R}$ stays near 1 — efficient exploration throughout.

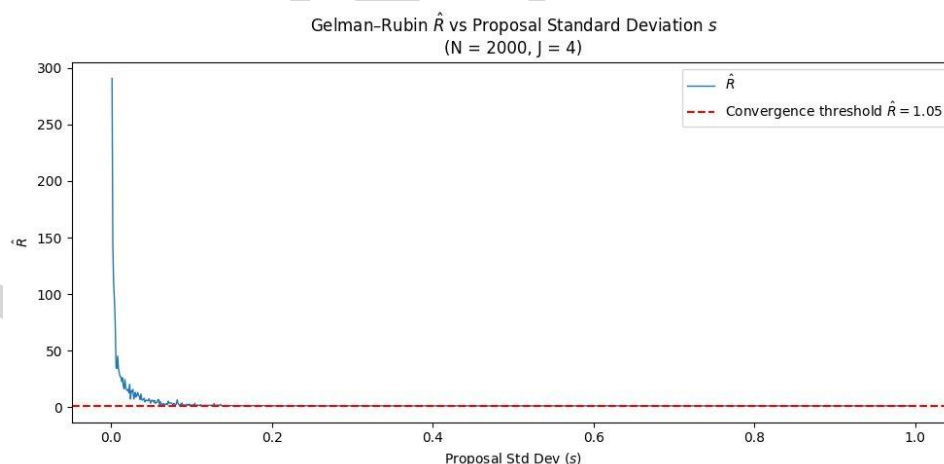


Figure 2. Gelman–Rubin $\hat{R}$ vs proposal standard deviation s ($N = 2,000$, $J = 4$). The dashed red line marks $\hat{R} = 1.05$. $\hat{R}$ collapses from very high values at small s to near 1 once s exceeds roughly 0.05.

4. Conclusion

The RWM algorithm reproduced the Laplace (0,1) distribution well when s was set appropriately. The histogram, KDE, and true PDF aligned closely (Figure 1), and the Monte Carlo estimates of the mean (0.0319) and standard deviation (1.4011) were close to their true values of 0 and $\sqrt{2} \approx 1.4142$.

Convergence depends heavily on s . At $s = 0.001$ the chains are stuck and $\hat{R}$ is well above 1.05; once s reaches moderate-to-large values $\hat{R}$ drops to near 1 (Figure 2). Python and R produced consistent results throughout, confirming the correctness of both implementations.

5. References

- Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A., & Rubin, D. B. (2013). *Bayesian Data Analysis* (3rd ed.). Chapman and Hall/CRC.
- Gelman, A., & Rubin, D. B. (1992). Inference from iterative simulation using multiple sequences. *Statistical Science*, 7(4), 457–472.
- Metropolis, N., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H., & Teller, E. (1953). Equations of state calculations by fast computing machines. *Journal of Chemical Physics*, 21(6), 1087–1092.
- Robert, C. P., & Casella, G. (2004). *Monte Carlo Statistical Methods* (2nd ed.). Springer.
- R Core Team. (2024). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. <https://www.R-project.org/>